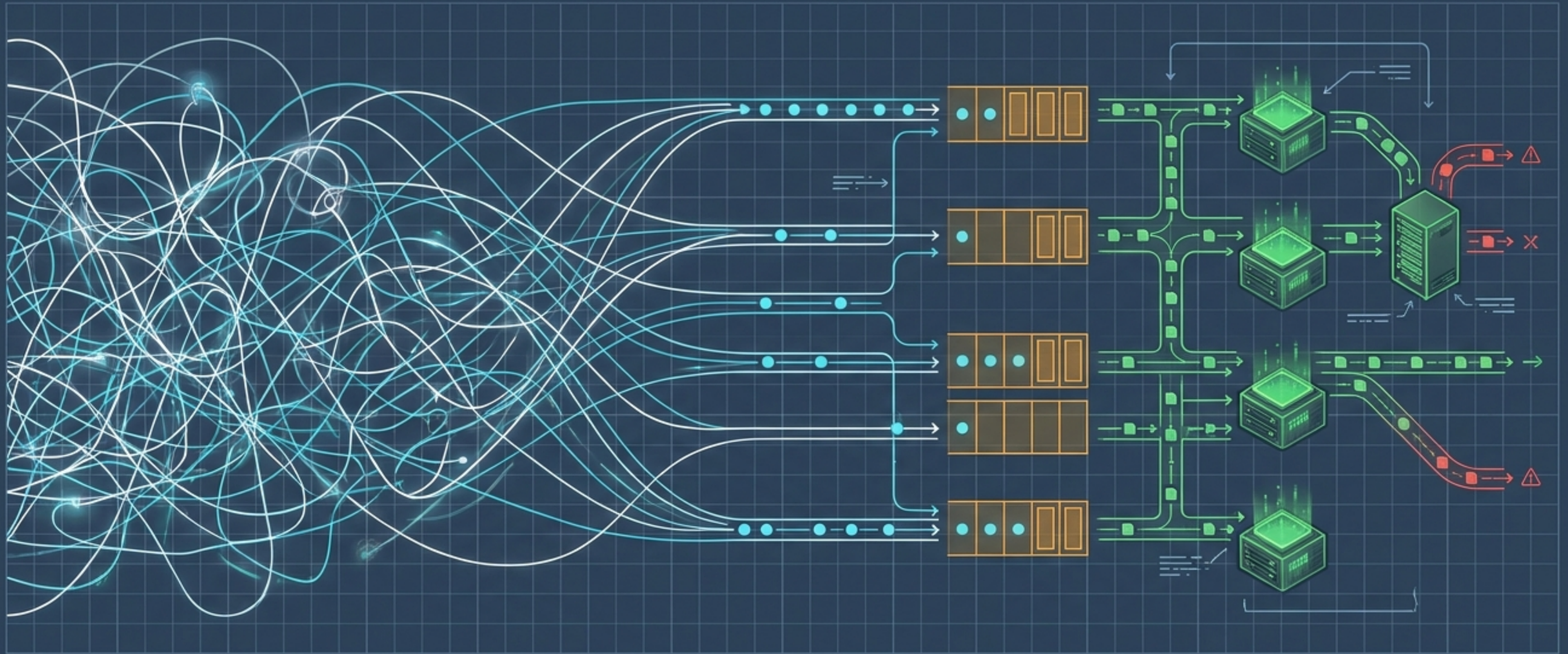
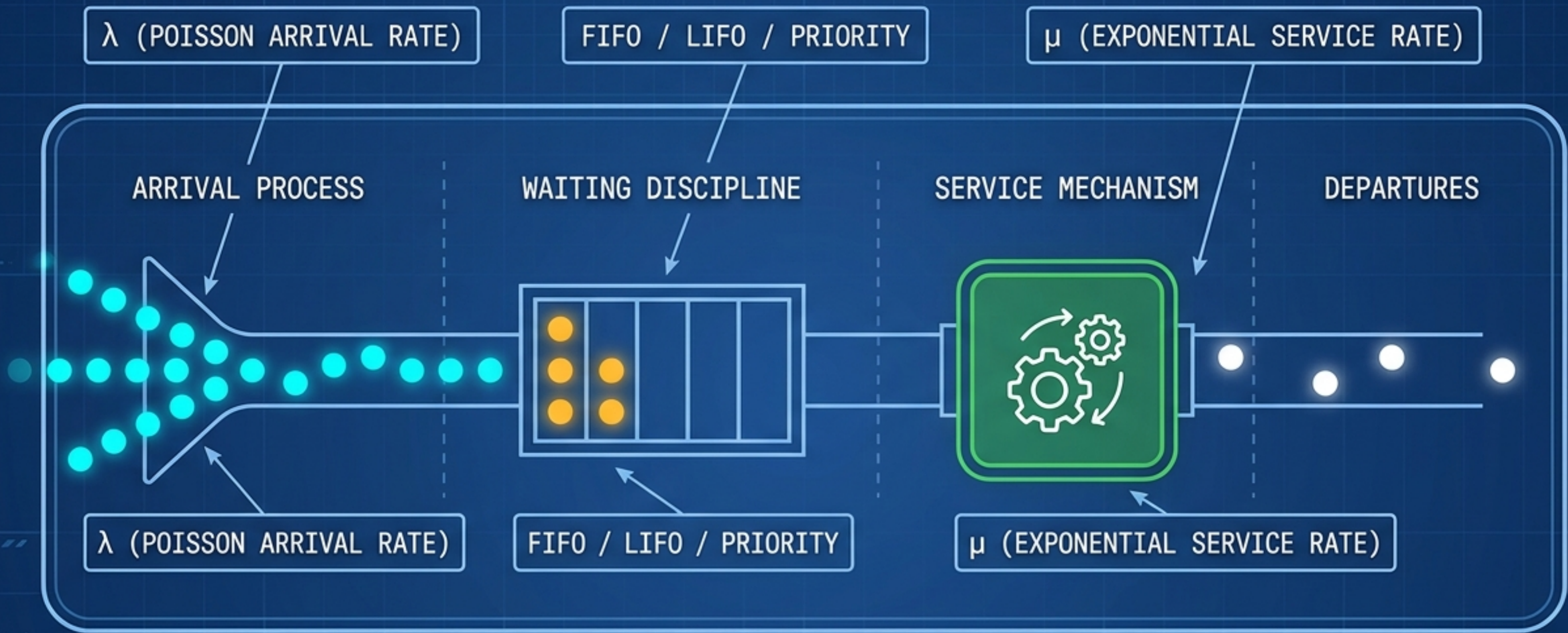




The Physics of Network Traffic

Queueing Systems, Performance Models, and Petri Nets





A QUEUE FORMS WHENEVER DEMAND FOR A NETWORK RESOURCE TEMPORARILY EXCEEDS ITS PROCESSING CAPACITY. CUSTOMERS (PACKETS, FLOWS, CALLS) WAIT FOR SERVERS (CHANNELS, PROCESSORS, LINKS).

Decoding Kendall's Notation

The Default Standard: $M / M / 1$



A / S / c / K / N / D

A: Interarrival-time distribution

S: Service-time distribution

c: Number of servers

K: System capacity

N: Population size

D: Queueing discipline

M = Markovian/Exponential
D = Deterministic
G = General

System Inventory (L)



Total number of customers
in the system

Queue Inventory (L_q)



Number of customers waiting

System Time (W)



Total response time

Queue Time (W_q)



Time spent waiting
before service

$$L = \lambda W$$

$$W = W_q + S$$

Why it matters:

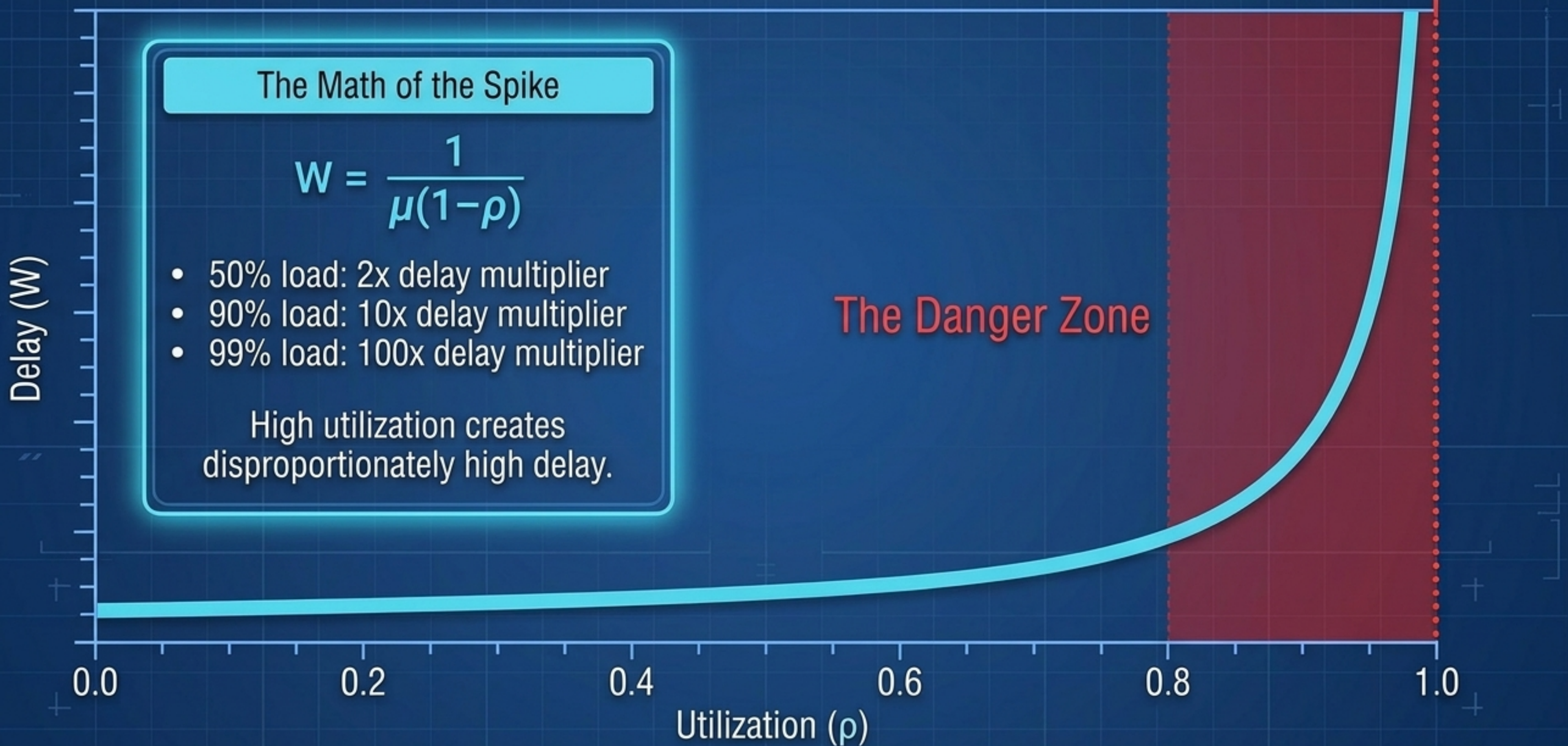
Derive delay simply from throughput and inventory.

$$\lambda = 1000 \text{ pkts/s}$$

$$L = 20 \text{ pkts}$$

$$W = 20/1000 = 0.02\text{s} \text{ (20 ms)}$$

The Nonlinear Reality of Utilization



The Baseline: Single-Server Dynamics

M / M / 1 Baseline



$$\rho = \lambda / \mu$$

$$L = \rho / (1 - \rho)$$

$$W = 1 / (\mu - \lambda)$$

$$\pi_n = (1 - \rho) \rho^n$$

M / G / 1 Reality

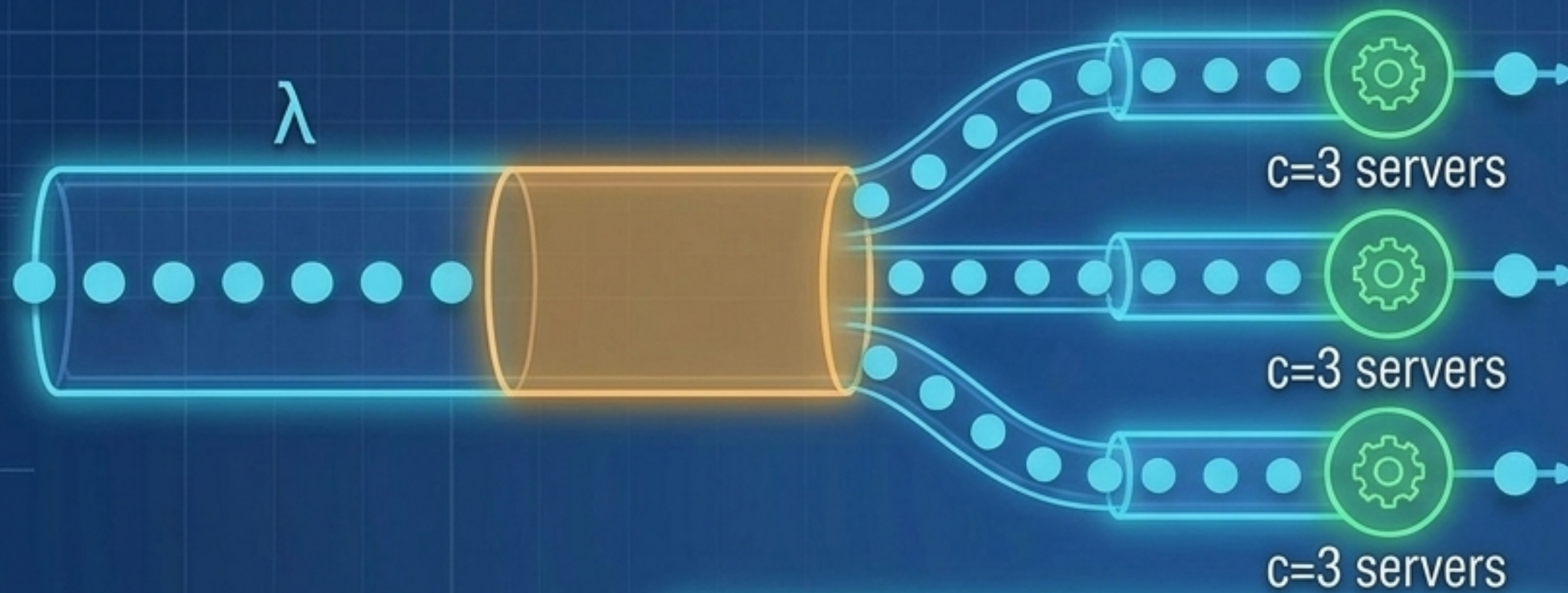


$$W_q = \frac{\lambda E[S^2]}{2(1 - \rho)}$$

Pollaczek-Khinchine Insight

Queueing delay depends on mean service time AND variability ($E[S^2]$). Heterogeneous traffic actively increases delay.

Scaling Horizontally: Multi-Server Queues (M / M / c)



Offered Load vs. Server Utilization

Offered Load (a) = λ/μ
Server Utilization (ρ) = $\lambda/c\mu$

Data Example

$\lambda = 8$ req/s
 $\mu = 3$ req/s
 $c = 3$ servers
Total Capacity ($c\mu$) = 9
Stable: $8 < 9$ ✓

All servers busy

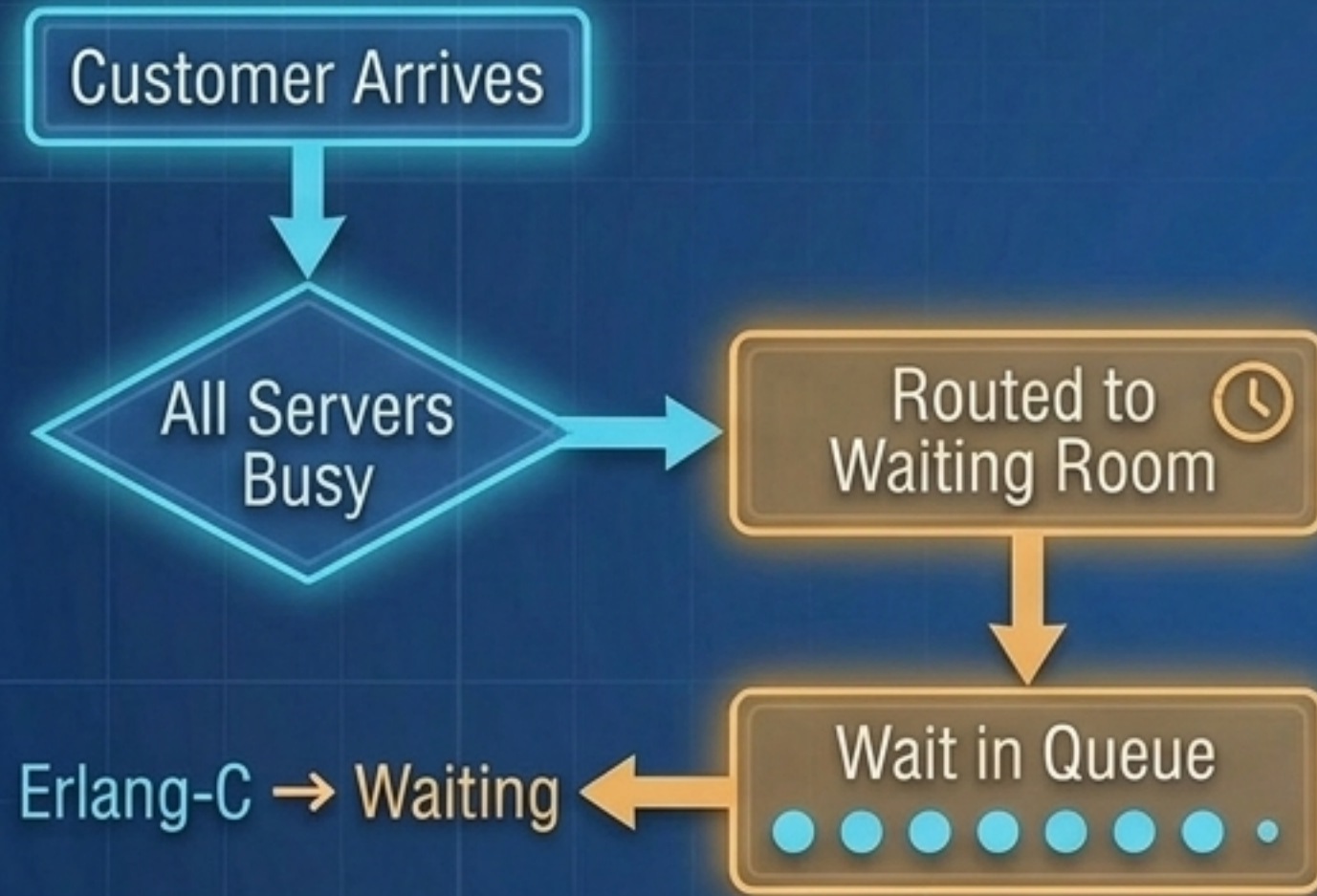
$$P_{\text{wait}} = \frac{\frac{a^c}{c!} * \left(\frac{1}{1-\rho}\right)}{\sum \left(\frac{a^n}{n!}\right) + \frac{a^c}{c!} * \left(\frac{1}{1-\rho}\right)}$$

Normalization factor

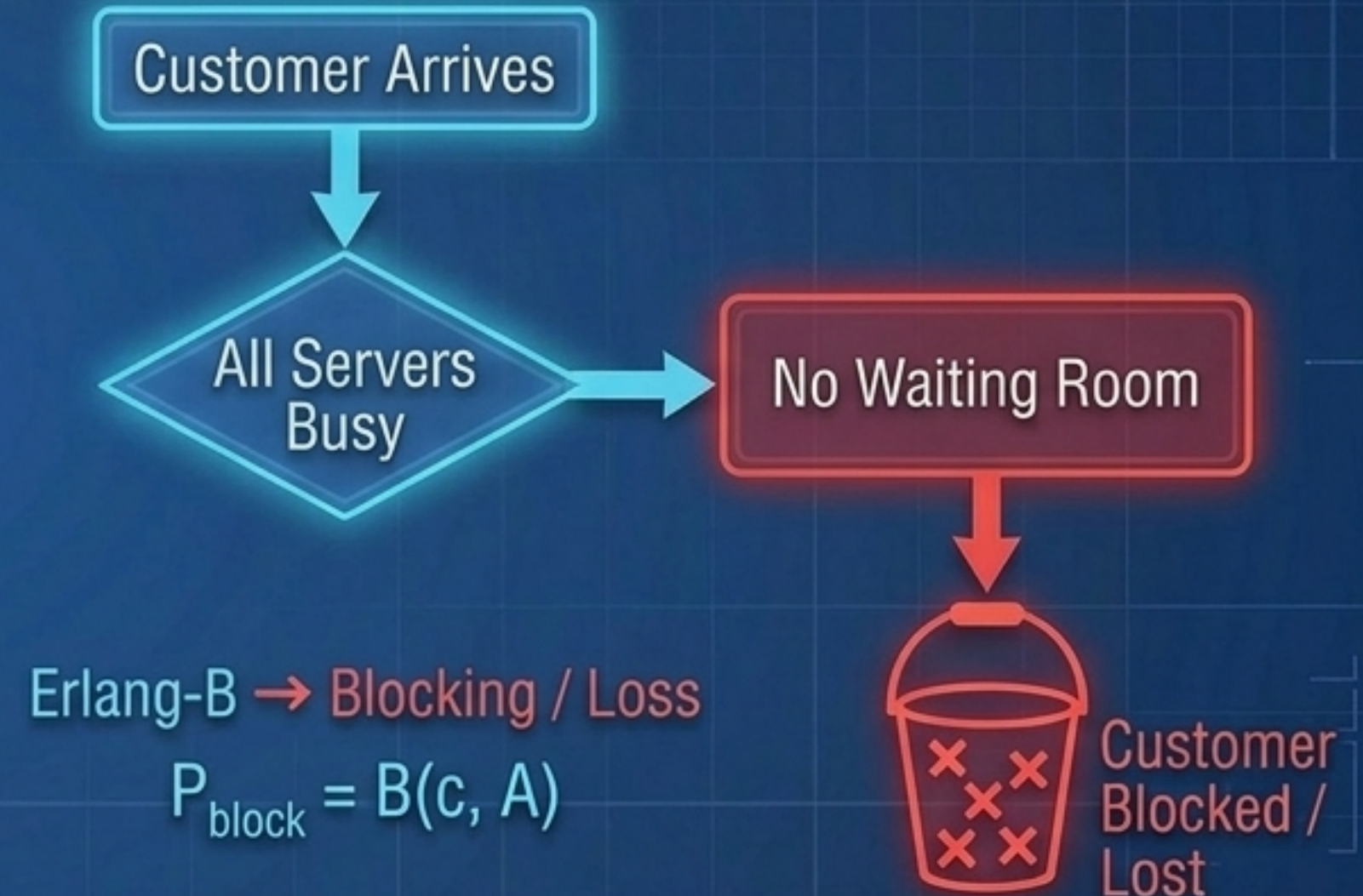
Key Takeaway: Adding servers increases total capacity and dramatically reduces waiting time, but as λ approaches $c\mu$, the system remains highly sensitive to load spikes.

The Zero-Wait Scenario: Erlang Loss Systems

Erlang-C (M / M / c)



Erlang-B (M / M / c / c)



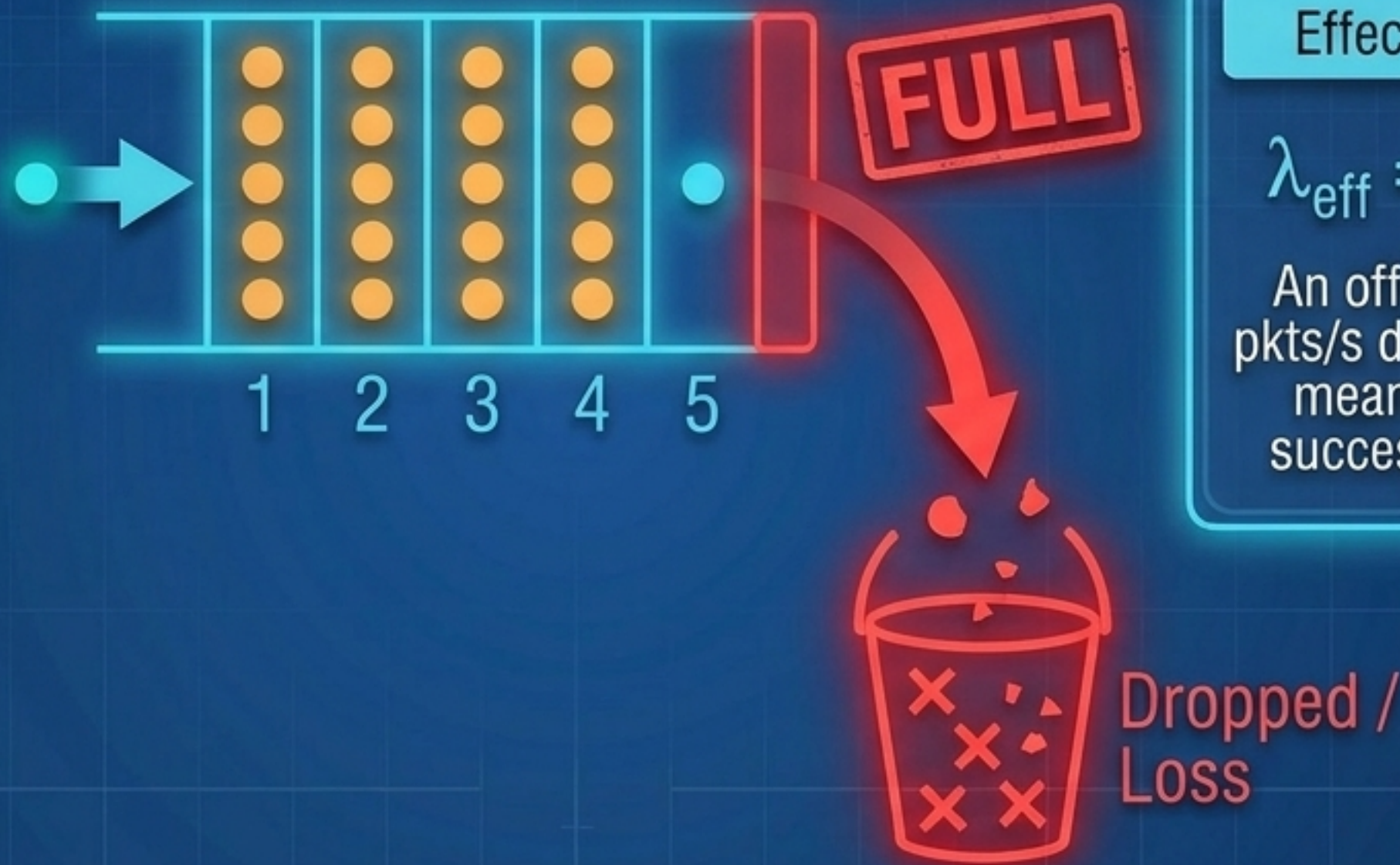
What is an Erlang? $A = \lambda E[S]$
2 calls/min $\times$ 5 min duration = 10 Erlangs (10 simultaneous active calls)

The Finite Reality: Limited Capacity (M / M / 1 / K)

Full System Probability

$$\pi_K = \frac{(1 - \rho)\rho^K}{1 - \rho^{K+1}}$$

Packet Loss Probability
(P_{loss}) = π_K



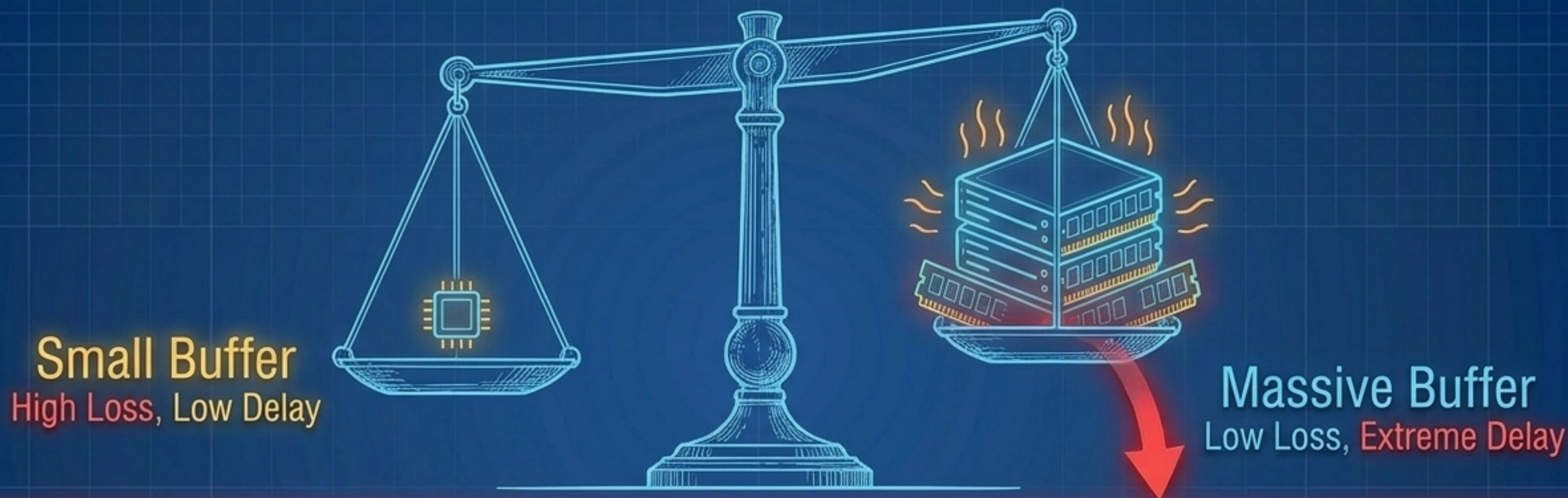
Effective Throughput

$$\lambda_{\text{eff}} = \lambda(1 - P_{\text{loss}})$$

An offered load of 1000
pkts/s does not mean 1000
mean 1000 pkts/s are
successfully processed.

Performance-based Dimensioning
If target $P_{\text{loss}} \leq 0.001$, solve for K to size memory correctly.

The Stability Paradox & Bufferbloat



⚠ Stability does NOT imply zero packet loss ⚠

A stable system ($\rho < 1$) with finite capacity will still drop packets. Conversely, over-provisioning buffer space ($K \rightarrow \infty$) eliminates loss but traps packets in the queue for massive durations—creating Bufferbloat.

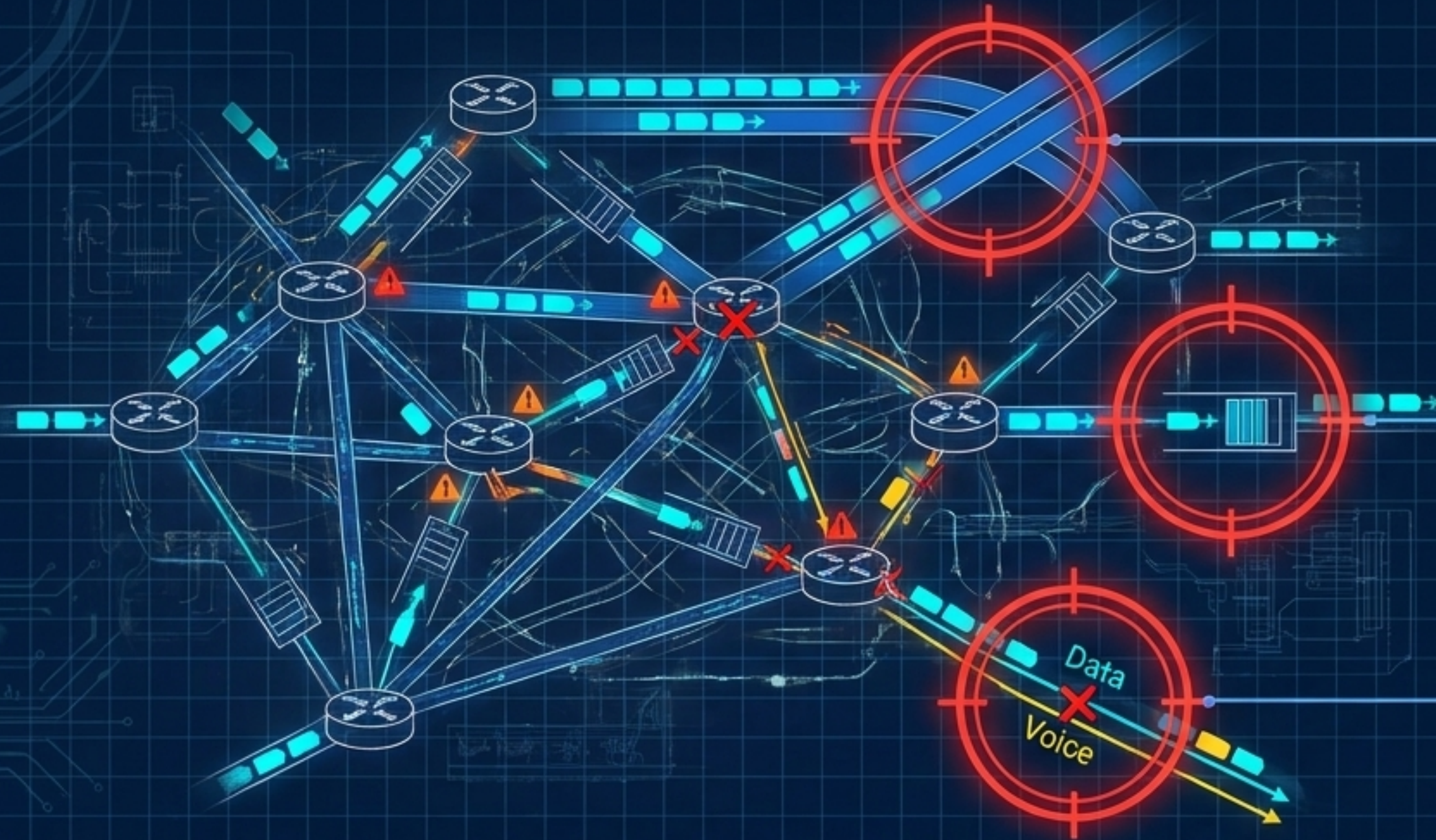
Network performance cannot be optimized using a single metric.
Efficiency, Delay, and Reliability must be balanced.

SYNTHESIS MATRIX: The Queueing Models Diagnostic Table

Model	Servers	Waiting Room (Capacity)	Key Metric	Primary Use Case
M / M / 1	1	∞	Delay (W)	Simple isolated processing
M / M / c	c	∞	Wait Prob (Erlang-C)	Server farms, parallel channels
M / M / c / c	c	0	Blocking Prob (Erlang-B)	Telephone networks, strict circuit switching
M / M / 1 / K	1	K	Packet Loss (P_loss)	Real-world routers, finite buffers

Different mathematical formalisms describing specific physical constraints.

Network Complexity: Beyond Simple Queues



Concurrency
Links transmitting simultaneously

Synchronization
Operation waiting for BOTH a channel and a buffer

Conflict
Voice and Data competing for single bandwidth token

Why Queueing Models Fall Short

Isolated models focus on the number of customers. They fail to capture **concurrency**, synchronization, and logical resource conflicts. For network reality, we need a **new structural language**.

The Anatomy of a Petri Net

Components & Definitions



1. Places (Circles)

Represent conditions or resources (e.g., Buffer space).



2. Tokens (Dots)

Represent the presence of a condition (e.g., Packets waiting).



3. Transitions (Bars)

Represent events (e.g., Packet transmission).



4. Arcs (Arrows)

Define relationships ($P \rightarrow T$ consumes, $T \rightarrow P$ produces).

System Dynamics: Firing Sequence

State 1: Token in input place (Transition enabled)



State 2: Transition fires

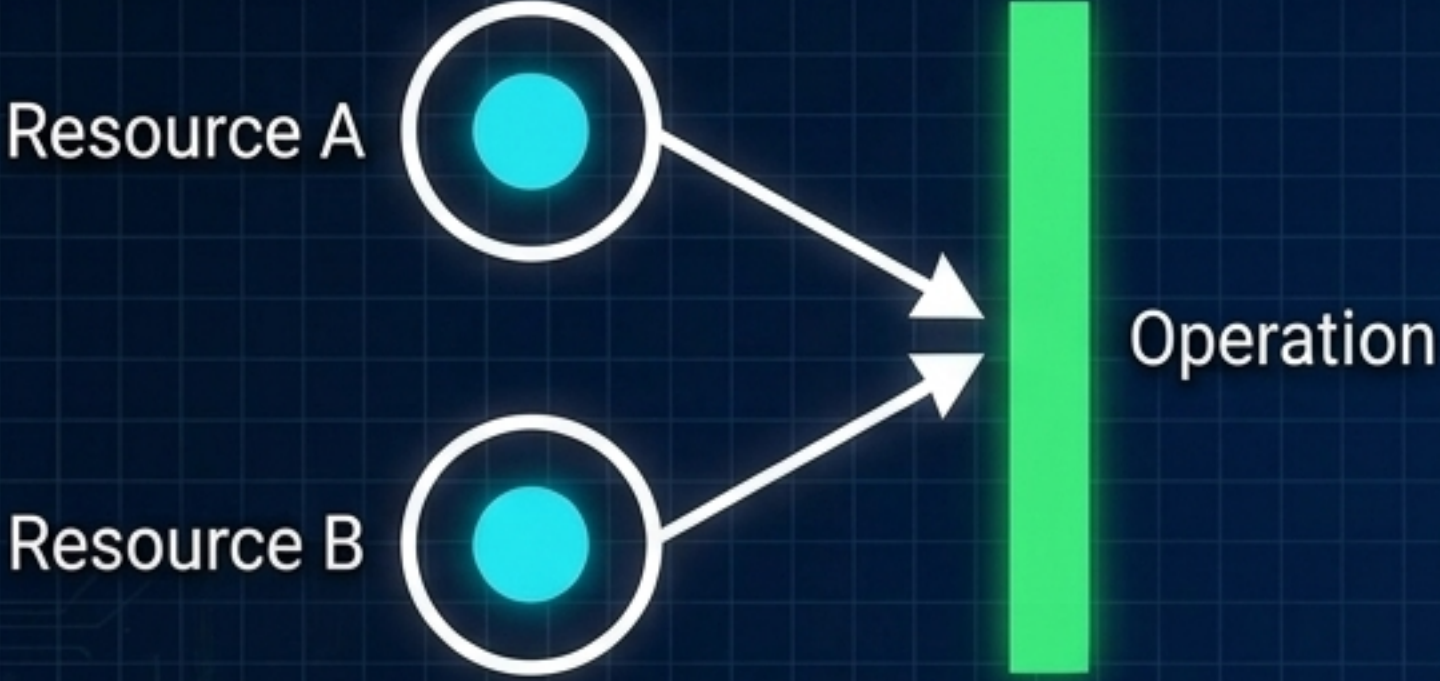
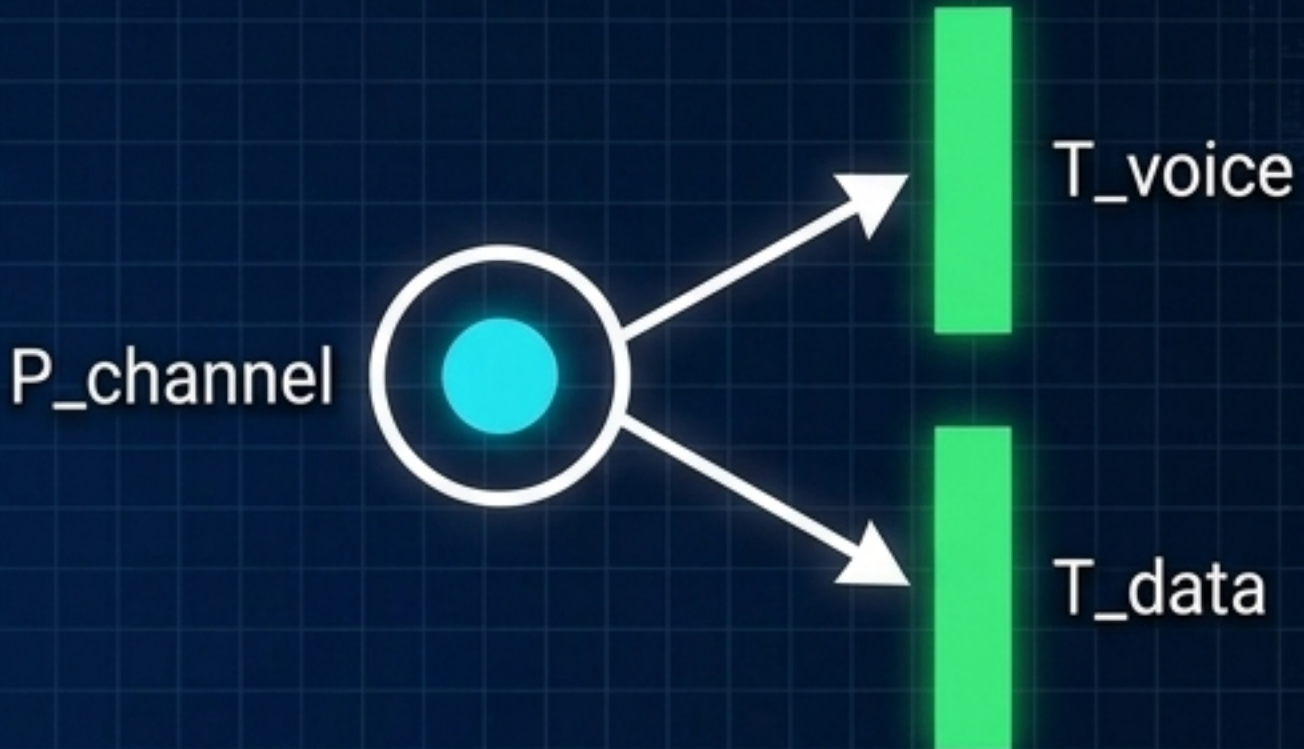


State 3: Token moved to output place



The State of the System = The Marking: $M = [m_1, m_2, \dots, m_m]$

Modeling the Chaos: Conflicts & Synchronization

Synchronization	Conflict (Resource Contention)
 Resource A and Resource B are places containing tokens. Arrows from both places point to a single transition labeled 'Operation'. Transition is enabled only when BOTH places have tokens. Models distributed operations requiring multiple simultaneous resources.	 P_channel is a place containing a token. Two arrows from this place point to two transitions labeled T_voice and T_data. One token enables competing transitions, forcing the system to resolve a deadlock or conflict.

Reachability

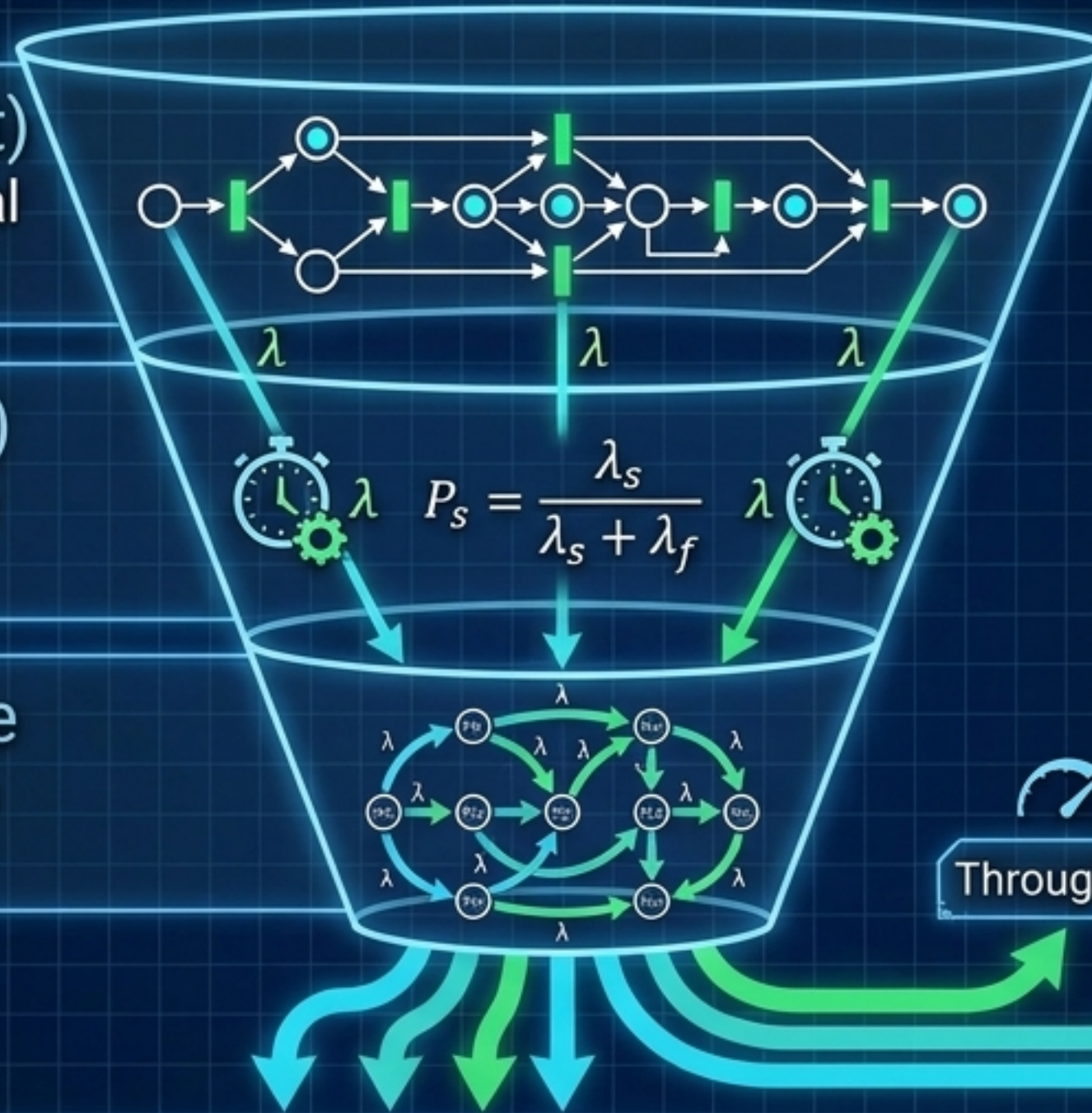
By modeling this algebraically ($M = M_0 + C\sigma$), we can mathematically predict deadlocks and buffer overflows.

Stochastic Petri Nets & Final Synthesis

System Structure (Petri Net)
Models concurrency and logical conditions.

Stochastic Timing (Rates)
Competing transitions resolve probabilistically.

Markov Chain State Space
Continuous-Time probabilistic state evolution.



The Architecture of Performance

Petri Nets define the system structure. Markov Chains define the probabilistic state evolution.
High utilization, finite buffers, and traffic variability interact to determine network survival.
Performance requires balancing efficiency, delay, and capacity simultaneously.